

Module 2 Question Pool

OMSCS 6250 Computer Networks

Lesson 2: Transport and Application Layers

Introduction to the Transport Layer

Q1. [MCQ]

Hosts A and B run the same application. The two hosts are in different networks, connected by multiple links and routers. IP is responsible for carrying the packets from host A to host B. The intermediate routers operate only on the IP headers of the packets.

What does the transport layer add that the network layer does not provide?

- **A.** Delivery to the right process, and reliable delivery, since IP reaches only the host and may lose data.
- **B.** Delay bounds, so that a real-time application gets each packet within a fixed time across the network.
- **C.** Delivery to the right host, since an IP address names the network and the transport layer names the host.
- **D.** Route selection, since the transport layer picks a less congested path through the routers for each packet.

Correct answer: **A**

Why: IP delivers each packet to a host, on a best-effort basis, with no promise of delivery or integrity. The transport layer adds port numbers, so the data reaches the right process on that host, and TCP adds reliable delivery. The transport layer runs only in the end systems. It cannot choose the path through the routers, and it cannot bound delay, because the packets still have to cross the network.

Multiplexing: Why Do We Need It?

Q2. [TF]

Host H runs two applications at the same time, a web browser and a music player. A packet addressed to one of these applications arrives at host H.

The destination IP address of the packet is sufficient for host H to deliver the packet to the correct application.

- True
- False

Correct answer: **False**

Why: An IP address identifies a host. The browser and the music player share H's address, so the address cannot tell them apart. The transport layer adds port numbers, and each application claims a port by opening a socket.

Connection Oriented and Connectionless Multiplexing and Demultiplexing

Q3. [MCQ]

Host B runs several applications that use UDP, and each application has its own socket. Host A sends a UDP segment to host B.

Which header fields does the transport layer at host B use to deliver the segment to the correct socket?

- **A.** The source IP address and source port, which tell host B which remote application sent the segment.
- **B.** The destination IP address and destination port, which are the same for every sender to that socket.
- **C.** The source and destination IP addresses and ports, the four fields identifying the socket.
- **D.** The destination port and the source IP address, so that each sending host reaches its own socket.

Correct answer: **B**

Why: A UDP socket is identified by a two-tuple, the destination IP address and destination port. One UDP socket serves every sender, so segments from different hosts reach the same socket, and the socket is chosen from the destination fields alone. The source fields serve as the return address for a reply. The four-tuple identifies a TCP socket, which belongs to one connection.

Q4. [MCQ]

A web server listens for TCP connections on port 80. Many clients are connected to the server at the same time. Every segment that these clients send to the server carries destination port 80.

How does the server deliver each arriving segment to the correct socket?

- **A.** It matches the source port alone, since each client's operating system picks a port no other client uses.
- **B.** It matches the source IP address alone, since each client host has an address that no other host uses.
- **C.** It matches the source and destination IP addresses and ports, since each connection has its own socket.
- **D.** It matches the sequence number, since each connection starts from its own random initial number.

Correct answer: **C**

Why: A TCP socket is identified by the four-tuple. Every client sends to the same destination IP address and port 80, so the source IP address and source port together separate the connections. Clients pick source ports independently, so two clients can pick the same port, and two connections from one client host share a source IP; either field alone can leave two connections looking alike. Sequence numbers order bytes within a connection, after the socket has been chosen.

Q5. [MCQ]

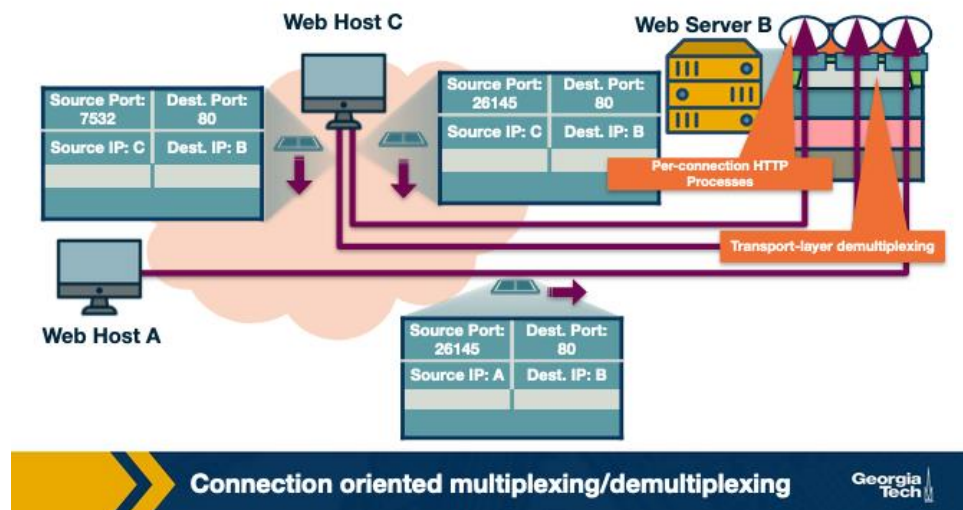


Figure: Multiple HTTP sessions to one web server (Module 2)

The figure shows three connections to web server B, which listens on port 80. Host C opens two of them, from source ports 7532 and 26145. Host A opens the third, from source port 26145. Suppose server B chose the socket from the source port, the destination IP address and the destination port, and ignored the source IP address.

Which of the three connections would reach the same socket?

- A. All three, since all three connections are sent to the same destination port, 80, on B.
- B. The two from host C, since they come from the same host and B sees them as one sender.
- C. None of them, since each connection is served by its own HTTP process on server B.
- D. The two with source port 26145, from hosts A and C; host C's 7532 connection stays apart.

Correct answer: **D**

Why: TCP identifies a socket by four fields. Without the source IP address, the connections from hosts A and C look the same: both carry source port 26145, destination IP address B and destination port 80, so their segments would reach one socket. Host C's two connections use different source ports, 7532 and 26145, so they stay apart, and that difference also keeps all three from merging. The HTTP processes read from sockets, so they cannot separate segments that have already reached one socket.

Q6. [MCQ]

A UDP server has a single socket, bound to IP address 192.0.2.7 and port 5060. Two datagrams addressed to this IP address and port arrive at the server. The first datagram comes from IP address 203.0.113.4, source port 41000. Shortly after, the second datagram comes from IP address 203.0.113.9, also from source port 41000.

What happens to the two datagrams at the server?

- A. Both datagrams reach the same socket, since UDP picks the socket from the destination IP and port.

- **B.** The second datagram is rejected, since UDP senders on two different hosts may not share a source port.
- **C.** The kernel creates a second UDP socket for the new sender, since each sender needs its own UDP socket.
- **D.** The second datagram is on hold until the application has finished its exchange with the first sender.

Correct answer: **A**

Why: A UDP socket is identified by the destination IP address and port alone. Datagrams with the same destination port reach the same socket even when they come from different hosts, so one socket serves both senders at once. Source ports are chosen by each sending host and need not differ across hosts. UDP creates no per-sender socket and keeps no per-sender state.

Q7. [MCQ]

A browser on a laptop opens six parallel TCP connections to one CDN server. All six connections go to the same server IP address and the same destination port, 443.

What allows the server to keep the six connections apart?

- **A.** The server keeps one socket for the laptop and separates the six streams by sequence number.
- **B.** The laptop sends each connection from its own IP address, so the six source IPs differ.
- **C.** Each connection has its own source port, so the six four-tuples differ in that field.
- **D.** The browser labels each HTTP request with a connection number, and the server reads it.

Correct answer: **C**

Why: Each TCP connection has its own socket, identified by the four-tuple. The six connections share the source IP address, the destination IP address and port 443, and the operating system gives each one its own source port, so the server holds six sockets. Sequence numbers order the bytes within one connection. Demultiplexing happens in the transport layer, before any HTTP request is read.

Q8. [TF]

Two UDP segments arrive at host B. Both segments carry the same destination IP address and the same destination port. The two segments come from two different source hosts.

The transport layer at host B delivers the two segments to two different sockets, one for each source host.

- True
- False

Correct answer: **False**

Why: A UDP socket is identified by the destination IP address and destination port only. Segments with the same destination port reach the same socket even when they come from different source hosts. One UDP socket serves every sender at once; a TCP socket, by contrast, belongs to one connection.

Q9. [MCQ]

A web server listens for TCP connections on port 80. Many clients are connected to the server at the same time. Suppose that the server identified its TCP sockets the way UDP identifies sockets, by the destination IP address and the destination port only.

What would go wrong?

- **A.** Segments from every connected client would reach one socket, so the connections could not be kept apart.
- **B.** New clients could not reach the listening socket, since their requests would match no socket on the server.
- **C.** Two clients that picked the same source port would be merged, while clients with distinct ports stayed apart.
- **D.** Segments would reach the right socket once the server had compared the sequence numbers they each carry.

Correct answer: **A**

Why: A TCP socket belongs to one connection. All clients of the web server send to the same destination IP address and port 80, so the two-tuple is identical across their connections, and every segment would land in one socket with no way to tell one client's byte stream from another's. New requests would still arrive, since they carry the server's address and port 80; the failure comes when the connections must be kept apart. A rule that reads only destination fields merges clients whatever their source ports, and sequence numbers order bytes within one connection only.

Q10. [MCQ]

A busy web server receives TCP connections from thousands of clients. Each page on the server holds dozens of small objects. The clients switch from persistent HTTP to non-persistent HTTP.

Why can this change overload the server?

- **A.** Each object now travels over UDP instead of TCP, so the server detects and resends lost objects on its own.
- **B.** Each object now needs a new destination port on the server, so the server can run out of port numbers.
- **C.** Each object now costs a new connection, socket and server state, and that work can rival serving the object.
- **D.** Each object now shares one socket with the other clients, so the server must sort out the mixed streams.

Correct answer: **C**

Why: With non-persistent HTTP, every request and response gets a new TCP connection, with its own socket and state, which is then closed. Every object therefore costs a connection setup, and on a busy server that overhead can rival the work of serving the content. The connections still use TCP and still arrive on port 80, where the server separates them by source IP address and source port. Each object gets a socket of its own, the opposite of sharing one socket among clients.

A Word About the UDP Protocol

Q11. [MCQ]

A video-call application sends its audio over UDP rather than TCP. Delay-sensitive applications such as this one choose UDP for several reasons.

Which of the following is NOT one of those reasons?

- A. UDP sends the first datagram at once, with no handshake, so no round trip is spent on setup.
- B. UDP applies no congestion control, so the sender does not slow down when the network is busy.
- C. UDP passes the data to the network layer as soon as the application hands it over to UDP.
- D. UDP delivers each datagram within a fixed delay, so the audio arrives in time to be played.

Correct answer: **D**

Why: UDP gives no delay guarantee, because the packets still have to cross the network and no protocol at the endpoints can bound that delay. UDP's real advantages for a call are that it sends with no connection setup, has no congestion control to slow the sender, and hands data to the network layer as soon as the application passes it down.

Q12. [TF]

A host builds a UDP segment that carries 100 bytes of application data.

The UDP header of the segment is 64 bits long and holds four fields: the source port, the destination port, the length and the checksum.

- True
- False

Correct answer: **True**

Why: The UDP header has four 16-bit fields: source port, destination port, length and checksum. Four fields of 16 bits make 64 bits, or 8 bytes, against TCP's 20.

Q13. [MCQ]

A local DNS server sends queries to other DNS servers on behalf of its clients. Each query and each answer is short. Most of these queries are carried over UDP rather than TCP.

What is the main reason DNS uses UDP for these queries?

- A. UDP sends the query at once with no handshake, so each lookup saves a full round trip.
- B. UDP retransmits a lost query sooner than TCP would, so a lookup recovers from loss faster.
- C. UDP's checksum repairs a damaged answer, so the resolver does not have to ask a second time.
- D. UDP applies congestion control to each query, which keeps a busy resolver from overload.

Correct answer: **A**

Why: TCP needs a three-way handshake before any data moves, which costs a round trip, while UDP sends the query at once. That saving is the main reason DNS usually runs over UDP. UDP is minimal: it retransmits nothing, its checksum only detects errors, and it applies no congestion control, so a lost query is resent by the application.

Q14. [MCQ]

An online multiplayer game is fast-paced. The game sends position updates 60 times per second. Each update replaces the one before it. These updates travel over UDP rather than TCP.

Why is UDP the more natural choice of transport protocol for these position updates?

- **A.** UDP applies a lighter congestion control than TCP, so the updates are slowed down less often.
- **B.** TCP would run a three-way handshake before each update, so each update would wait one round trip.
- **C.** The game accepts some lost updates, and TCP's resends of lost updates would add delay.
- **D.** UDP resends a lost update sooner than TCP would, so a lost position reaches the players faster.

Correct answer: C

Why: The game sends a new position 60 times per second, so it can accept a lost update and carry on with the next one. TCP retransmits unacknowledged packets and applies congestion control, and both add delay. UDP leaves out retransmission and congestion control entirely, so a lost update stays lost. TCP's three-way handshake runs once per connection, when the connection opens.

Q15. [MCQ]

A VoIP application carries a voice call between two hosts. The sender sends a small audio packet every 20 milliseconds. The receiver must play each audio packet at a fixed moment, soon after the packet is due to arrive. The application runs over TCP instead of UDP.

Which property of TCP hurts the call?

- **A.** A three-way handshake runs before each audio packet, adding one round trip to each sample.
- **B.** Its congestion control discards audio packets when the path is busy, so the call loses words.
- **C.** A lost segment is resent, and the resent audio arrives after the moment it had to be played.
- **D.** Its 20-byte header is too large to carry the small audio payloads that voice codecs produce.

Correct answer: C

Why: TCP retransmits lost segments and delivers data in order. A resent audio packet arrives after its playing time has passed, and the packets behind it wait too. The handshake happens once, when the connection opens, and congestion control slows the sender without discarding any data. TCP's 20-byte header is larger than UDP's 8 bytes, but it carries small payloads as well as large ones.

Q16. [MCQ]

A user uploads a 4 GB ZIP archive over the Internet to a cloud storage service. The upload runs over TCP rather than UDP.

Why is TCP the more natural choice of transport protocol for this upload?

- **A.** A missing piece corrupts the archive, and TCP resends lost segments and keeps the bytes in order.
- **B.** UDP carries no checksum, so a segment damaged on the way would reach the server unnoticed.
- **C.** TCP's handshake reserves bandwidth along the path, so the upload keeps a steady rate throughout.
- **D.** TCP's per-connection state lets the server handle more uploads at once than it could handle over UDP.

Correct answer: **A**

Why: A file with missing pieces is corrupted. TCP resends lost segments and delivers the data in order, so the application does not have to build that itself. UDP also carries a checksum, which detects a damaged segment without repairing it. TCP runs only in the end systems, so it cannot reserve bandwidth or guarantee a rate across the network, and its per-connection state is a cost: a server supports far more active clients over UDP than over TCP.

The TCP Three-Way Handshake

Q17. [TF]

A client opens a TCP connection to a server with the three-way handshake. The client has sent its SYN segment. The client has then received the server's SYNACK segment.

The next segment that the client sends to the server has its SYN bit set to 0.

- True
- False

Correct answer: **True**

Why: In the third step the client acknowledges the SYNACK. The SYN bit is set only in the first two segments, the client's request and the server's reply, so the third carries SYN set to 0.

Q18. [MCQ]

A client opens a TCP connection to a server. The client sends a SYN segment with its initial sequence number, `client_isn`. The server answers with a SYNACK segment.

What value does the server put in the acknowledgment number field of the SYNACK segment?

- **A.** `server_isn`, the random initial sequence number the server picked for its own direction.
- **B.** `client_isn + 1`, one more than the initial sequence number the client sent in its SYN.
- **C.** Zero, since the server has not yet received any application data from the client.
- **D.** `client_isn`, the same number the client sent, so that both sides confirm the same value.

Correct answer: **B**

Why: The SYNACK acknowledges the client's SYN by setting the acknowledgment field to `client_isn + 1`, the next number the server expects. `server_isn` goes in the SYNACK's sequence number field. The acknowledgment is sent even though no application data has arrived, since the SYN itself is being acknowledged.

Q19. [TF]

Two hosts close a TCP connection. Soon after, the two hosts open a new TCP connection between the same two port numbers. A delayed segment from the old connection is still in the network. Each side of the new connection picks its initial sequence number at random.

The random choice makes it unlikely that the delayed segment is accepted as data of the new connection.

- True
- False

Correct answer: **True**

Why: Old segments can outlive the connection that created them. If both sides started from zero, a straggler from the earlier connection could carry sequence numbers that the new connection would accept. Random starting points make that very unlikely.

Reliable Transmission

Q20. [MCQ]

A sender and a receiver use the Go-back-N protocol. The sender transmits a window of packets. One packet in the middle of the window is lost in the network. The packets that follow the lost packet reach the receiver.

Which statement describes how Go-back-N recovers from this loss?

- **A.** The receiver acknowledges each packet that arrives, and the sender resends only the lost packet.
- **B.** The receiver discards the later packets, and the sender resends only the packet that was lost.
- **C.** The receiver acknowledges the last in-order packet, and the sender resends from the lost one on.
- **D.** The receiver buffers the later packets, and the sender resends from the lost packet onward.

Correct answer: **C**

Why: A Go-back-N receiver keeps no out-of-order packets. It acknowledges the last packet it received in order, and the sender resends every packet from the lost one onward, including those that had arrived. Acknowledging each packet, buffering later packets and resending only the lost one are features of Selective Repeat.

Q21. [MCQ]

A TCP sender has several segments in flight to a receiver. One segment is lost in the network. The segments that follow the lost segment reach the receiver. For each of these later segments, the receiver sends a duplicate ACK.

How many duplicate ACKs must the sender receive before it retransmits the lost segment without waiting for its retransmission timer to expire?

- A. One duplicate ACK.
- B. Two duplicate ACKs.
- C. Three duplicate ACKs.
- D. Four duplicate ACKs.

Correct answer: **C**

Why: On three duplicate ACKs the sender concludes the segment is lost and retransmits it at once. This is fast retransmit. The threshold is three rather than one because reordering in the network can produce a duplicate ACK or two with nothing lost.

Q22. [MCQ]

A sender and a receiver use the Go-back-N protocol. The sender transmits packets 1 to 10. Packets 1 to 6 arrive at the receiver in order. Packet 7 is lost in the network. Packets 8, 9 and 10 then arrive at the receiver.

What does the receiver send back to the sender each time one of packets 8, 9 and 10 arrives?

- A. An ACK for packet 6, the last packet it received in order.
- B. A negative ACK for packet 7, asking for that one packet only.
- C. One cumulative ACK for packet 10, since 8 to 10 arrived intact.
- D. A separate ACK for packet 8, then for 9, then for 10, as each arrives.

Correct answer: **A**

Why: A Go-back-N receiver acknowledges only the most recent packet it received in order. That packet is 6, so each arrival of 8, 9 and 10 brings another ACK for 6. Go-back-N has no negative ACK, and a separate ACK for each packet is what Selective Repeat sends. A cumulative ACK for 10 would claim that packet 7 had arrived.

Q23. [MCQ]

A sender and a receiver use the Go-back-N protocol. The sender transmits packets 1 to 10. Packets 1 to 6 arrive at the receiver in order. Packet 7 is lost in the network. Packets 8, 9 and 10 then arrive at the receiver.

What does the receiver do with packets 8, 9 and 10?

- A. It keeps them in its buffer until packet 7 arrives, then delivers 7 to 10 in order.
- B. It discards them, because they arrived out of order, after the gap at packet 7.
- C. It passes them to the application at once, and passes on packet 7 when it arrives.
- D. It keeps packet 10, the most recent packet, and discards packets 8 and 9.

Correct answer: **B**

Why: A Go-back-N receiver discards any packet that arrives out of order. Packets 8, 9 and 10 arrive after the gap at packet 7, so the receiver drops them, and the sender later resends everything from packet 7 onward. Buffering them until packet 7 arrives is Selective Repeat's behavior, and passing them up at once would deliver data out of order. What the receiver tracks is the last packet received in order, packet 6.

Q24. [MCQ]

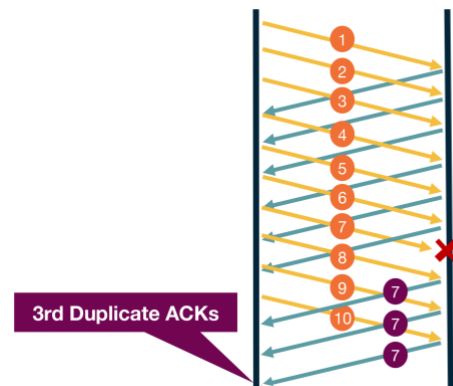


Figure: Packet 7 lost, followed by three duplicate ACKs (Module 2)

In the figure, a TCP sender transmits packets to a receiver, and packet 7 is lost. Packets 8, 9 and 10 reach the receiver. For each of these three packets, the receiver sends a duplicate ACK carrying 7, the number of the packet it still expects.

What does the sender do when the third of these duplicate ACKs arrives?

- **A.** It waits for its retransmission timer to expire, and only then resends packet 7.
- **B.** It resends packet 7 at once, without waiting for its retransmission timer to expire.
- **C.** It resends packets 7, 8, 9 and 10, since the receiver has discarded everything after 6.
- **D.** It sends new data after packet 10, since the ACKs show the later packets have arrived.

Correct answer: **B**

Why: Three duplicate ACKs tell the sender that packet 7 is lost while later packets still arrive. The sender retransmits packet 7 at once, without waiting for its timer; this is fast retransmit. TCP resends only the missing segment, whereas Go-back-N would resend everything after it as well. The duplicate ACKs all carry 7, which tells the sender that the receiver is still missing packet 7.

Transmission Control

Q25. [MCQ]

Three hosts each run an application that sends a large file across the same bottleneck link. All three hosts use TCP, and the three connections have the same RTT. The hosts do not know the capacity of the link, and they do not know how many other senders share it. Over time, each host still ends up with roughly an equal share of the link.

How do the hosts arrive at a fair share of the link?

- **A.** The hosts exchange control messages with one another and agree on how to divide the link's capacity.
- **B.** Each application measures the link by itself and picks its own sending rate, with no help from TCP.

- **C.** TCP at each host adds to its window steadily and halves it on loss; that rule evens out the shares.
- **D.** Signals from the application, transport and network layers are combined to give each host its share.

Correct answer: **C**

Why: TCP's AIMD rule produces the fair share. Additive increase raises every connection's window by the same amount, and multiplicative decrease cuts the larger window by more, so each round of losses shrinks the gap. Each host works alone, inferring congestion from its own lost packets, with no messages from the other hosts, the network or the application. Rate control lives in the transport layer because an application acting alone cannot reason about fairness.

Flow Control

Q26. [MCQ]

Host A sends a file to host B over a TCP connection. Host B sends acknowledgments back to host A. Each acknowledgment carries a value called the receive window, `rwnd`.

What does `rwnd` measure?

- **A.** The rate the network path can carry without loss, as the sender measures it in slow start.
- **B.** The bytes the sender has sent but not yet seen acknowledged, the data still in flight.
- **C.** The free space in host B's buffer, which is how much more data host B can accept now.
- **D.** The data sitting in host B's buffer that the application at host B has not yet read.

Correct answer: **C**

Why: `rwnd` is the space remaining in the receive buffer: the buffer size minus the data received but not yet read. It tells the sender how much the receiver can accept now. The data waiting to be read fills the rest of the buffer. The sender keeps its data in flight, `LastByteSent` minus `LastByteAked`, within `rwnd`, and the path's capacity is what the congestion window probes.

Q27. [TF]

Host A sends a file to host B over a TCP connection. The receive buffer at host B fills up, and host B advertises a receive window of zero, `rwnd = 0`. Host A stops sending. Host B has no data of its own to send to host A.

While `rwnd` is zero, host A keeps sending one-byte segments to host B, so that host A learns when buffer space becomes free.

- True
- False

Correct answer: **True**

Why: With `rwnd` at zero the sender stops, and if the receiver has nothing to send, no segment would carry news that space has reopened. TCP avoids this by having the sender keep sending one-byte segments. The receiver acknowledges each, and every acknowledgment carries the current window, so the sender learns when space reopens.

Q28. [MCQ]

Host A sends a file to host B over a TCP connection. The receive buffer at host B fills up. Host B advertises a receive window of zero, and host A stops sending. Host B has no data of its own to send to host A. Later, the application at host B reads all the data in the receive buffer.

If TCP had no extra mechanism for this case, why would host A never resume sending?

- **A.** Host B reports its window only on segments it sends, and it now has no segment to send.
- **B.** Host A resumes when a timeout expires, and its timer is stopped while the window is zero.
- **C.** Host B keeps reporting the old window, since rwnd is fixed when the connection is set up.
- **D.** Host B's window grows only when new data arrives from host A, and host A has stopped sending.

Correct answer: **A**

Why: The receiver advertises rwnd only in the segments and acknowledgments it sends. Once host B has drained its buffer the window has reopened, but host B has nothing to send, so no segment carries the new value to host A, and host A waits. The window changes throughout the connection, growing as host B's application reads and shrinking as data arrives. The stall comes from the missing advertisement rather than from any timer; TCP breaks it by having the sender keep sending one-byte segments, each of which host B acknowledges with the current window.

Congestion Control Introduction

Q29. [MCQ]

A packet travels from a source host to a destination host through several routers. The packet passes through the first four routers. The buffer at the fifth router is full, and this router drops the packet. The source host later retransmits the packet.

Why does a drop at the fifth router cost the network more than a drop at the first router would?

- **A.** The links the packet already crossed spent capacity on it, and that work now produces nothing.
- **B.** The sender must send the packet again, so some capacity carries the same data a second time.
- **C.** The fifth router's queue grows by that packet, so the packets behind it wait longer to leave.
- **D.** The fifth router reports the loss back up the path, so control messages add to the path's load.

Correct answer: **A**

Why: A packet dropped late has already used capacity on every link it crossed, and that capacity could have carried useful data. This is the most serious cost of congestion; taken far enough it becomes congestion collapse, where every link is busy and almost nothing useful gets through. A retransmission follows a drop at any router, the first one included, so it cannot explain why a late drop costs more. A dropped packet never joins the queue, and routers leave loss detection to the sender.

What are the goals of congestion control?

Q30. [TF]

On a network, most flows are short and finish within a few round trips. A few flows are long. A congestion control algorithm on this network brings a flow to its fair share only after the flow has run for many round trips.

This algorithm is unfair to most of the flows that use the network.

- True
- False

Correct answer: **True**

Why: A short flow that converges slowly finishes before it ever reaches its fair share. Since most flows are short, an algorithm that is fair only in the long run treats most flows unfairly. This is why fast convergence is one of the four goals of congestion control.

Congestion control flavors: E2E vs Network-assisted

Q31. [MCQ]

Congestion control can follow one of two approaches: end-to-end, or network-assisted. A TCP sender has to reduce its sending rate when the network becomes congested. The routers on the path can observe their own queues filling up. The sender cannot observe these queues.

Which statement about the two approaches is correct?

- A. End-to-end control relies on routers, which send explicit feedback such as ICMP source quench.
- B. Classic TCP uses network-assisted control, and ICMP source quench is how routers signal it.
- C. ECN is an end-to-end method, because the router's mark reaches the sender via the receiver.
- D. Classic TCP uses end-to-end control, and the hosts infer congestion from the loss of packets.

Correct answer: **D**

Why: In end-to-end control the network says nothing, and the hosts infer congestion from what they observe; the earliest TCP used packet loss. In network-assisted control routers tell the sender. ICMP source quench was the historical example and is no longer used. ECN is explicit feedback from a router, even though the mark reaches the sender through the receiver.

Q32. [TF]

A router on the path of a TCP connection supports ECN, and so do the two hosts of the connection. The queue at the router starts to fill.

The router marks a passing packet instead of dropping it, so the TCP sender learns of congestion before any packet is lost.

- True
- False

Correct answer: **True**

Why: With ECN a congested router marks a bit in the IP header instead of dropping the packet. The receiver echoes the mark back to the sender in an acknowledgment, and the sender reduces its window as it would for a loss. Congestion is signaled before any packet is lost.

How a host infers congestion? Signs of congestion

Q33. [MCQ]

A TCP sender cannot see packets being dropped inside the network. Instead, it infers loss from two signals: a retransmission timeout, and the arrival of three duplicate ACKs.

Which statement about these two signals is correct?

- **A.** A router sends the duplicate ACKs when it drops a segment, so three of them confirm the drop.
- **B.** A timeout confirms the segment was dropped, because a delayed segment would be ACKed before the timer expires.
- **C.** Both are indirect signals: a segment may be delayed or reordered, or its ACK may be lost, so the sender cannot be certain it was dropped.
- **D.** The sender waits until both signals occur for the same segment before it concludes that the segment was lost.

Correct answer: **C**

Why: Both signals are inferences. A timeout fires when no ACK arrives in time, but the segment may only be delayed, or it may have arrived and its ACK been lost; TCP calls these spurious timeouts. Duplicate ACKs mean the receiver got later segments but not the expected one, which usually means a loss but can also come from reordering. The receiver sends the duplicate ACKs, and either signal alone triggers a retransmission.

How Does a TCP Sender Limit the Sending Rate?

Q34. [TF]

A TCP sender has a congestion window of $cwnd = 40$ segments. The receiver advertises a receive window of $rwnd = 25$ segments.

The sender can have at most 25 unacknowledged segments outstanding.

- True
- False

Correct answer: **True**

Why: The sender must respect both windows, one protecting the network and one protecting the receiver, so its unacknowledged data is bounded by the smaller. Here that is $rwnd$, 25 segments.

Congestion Control at TCP - AIMD

Q35. [MCQ]

A TCP sender can detect a loss in two ways: by receiving three duplicate ACKs, or by a retransmission timeout. TCP treats three duplicate ACKs as the milder of these two loss events.

Why does TCP treat three duplicate ACKs as the milder loss event?

- A. Duplicate ACKs arrive sooner than a timeout fires, so the loss is caught while cwnd is small.
- B. Duplicate ACKs show later segments still reach the receiver; a timeout shows no ACK came back.
- C. A timeout comes from flow control at the receiver, while duplicate ACKs come from the routers.
- D. Duplicate ACKs show the loss was at the receiver, whose buffer clears faster than a router's.

Correct answer: **B**

Why: The receiver sends a duplicate ACK only when a later segment reaches it, so duplicate ACKs show that packets are still getting through. A timeout means no acknowledgment arrived at all, so the path may be broken. Duplicate ACKs come from the receiver and report only a gap in the sequence. The severity of a signal depends on what it shows about the path, whichever signal arrives first.

Q36. [MCQ]

A TCP sender has a congestion window, cwnd, of 16 packets. The sender then receives three duplicate ACKs.

What is the value of cwnd immediately after the three duplicate acknowledgments are received?

- A. 1 packet
- B. 8 packets
- C. 15 packets
- D. 17 packets

Correct answer: **B**

Why: Three duplicate ACKs are TCP's mild loss event, and it halves cwnd: 16 becomes 8. A reset to one packet is TCP's response to a timeout. The decrease is multiplicative, not a step of one packet, and cwnd does not keep growing once loss is detected.

Q37. [MCQ]

A TCP sender can detect a loss in two ways: by receiving three duplicate ACKs, or by a retransmission timeout.

How does the sender change its congestion window, cwnd, in each of these two cases?

- A. Three duplicate ACKs halve cwnd; a timeout resets cwnd to the initial window.
- B. Three duplicate ACKs halve cwnd; a timeout doubles cwnd to probe for capacity.
- C. Three duplicate ACKs leave cwnd unchanged; a timeout halves cwnd to ease the congestion.
- D. Three duplicate ACKs reset cwnd to the initial window; a timeout halves cwnd.

Correct answer: **A**

Why: TCP treats three duplicate ACKs as mild congestion and halves the window. It treats a timeout as severe and resets the window to its initial size. What doubles on a timeout is the timeout interval. Three duplicate ACKs signal a loss, and TCP reduces cwnd on every loss event.

Q38. [MCQ]

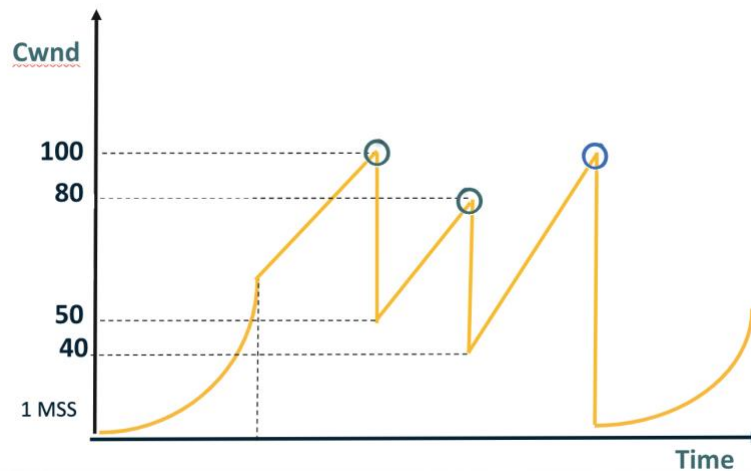


Figure: TCP cwnd over time (Module 2)

The figure shows the congestion window, cwnd, of a TCP sender over time. At one point, cwnd grows to 100 and then drops to 50 in one step.

Which event explains this drop?

- **A.** A retransmission timeout, since a timeout is the loss event TCP answers by halving cwnd.
- **B.** A new rwnd of 50 from the receiver, which caps cwnd at the receiver's free buffer space.
- **C.** Three duplicate ACKs, since that is the loss event TCP answers by halving cwnd.
- **D.** The end of slow start, since cwnd falls back to ssthresh the first time it crosses it.

Correct answer: **C**

Why: A drop from 100 to 50 is a halving, TCP's response to three duplicate ACKs. A timeout would reset cwnd to the initial window. The receive window caps how much the sender may have in flight, and cwnd changes only with ACKs and loss events. Slow start ends by switching to additive increase, with no drop.

Q39. [MCQ]

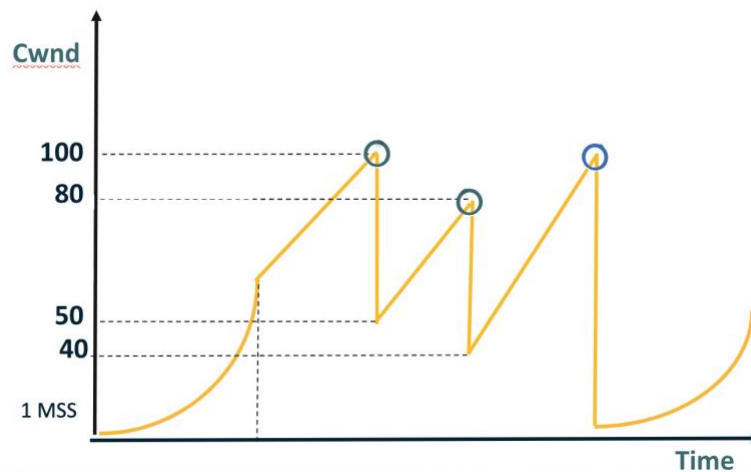


Figure: TCP cwnd over time (Module 2)

The figure shows the congestion window, cwnd, of a TCP sender over time. In the third climb of the plot, the window reaches about 100. The window then drops in one step to the bottom of the scale, far below 50.

Which event explains this drop?

- **A.** The end of congestion avoidance, since TCP restarts slow start after each peak it reaches.
- **B.** Three duplicate ACKs, which TCP treats as the severe signal and answers with a reset.
- **C.** A new rwnd of 1 from the receiver, which caps the sender at one segment in flight.
- **D.** A retransmission timeout, on which TCP resets cwnd to the initial window size.

Correct answer: **D**

Why: A drop to the bottom in one step is a reset, TCP's response to a timeout, when no acknowledgment arrives at all. Three duplicate ACKs are the mild signal, and TCP halves the window for them. Slow start restarts only after a timeout, and a small rwnd would limit sending while leaving cwnd where it was.

Q40. [MCQ]

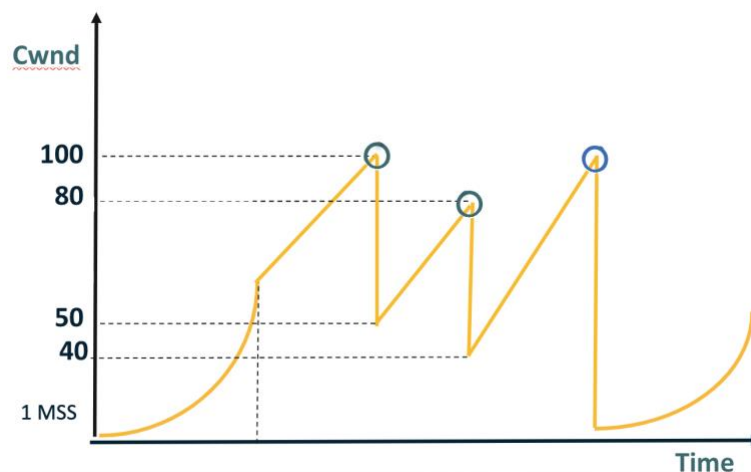


Figure: TCP cwnd over time (Module 2)

The figure shows the congestion window, *cwnd*, of a TCP sender over time. Three duplicate ACKs cut *cwnd* from 100 to 50 segments.

What value does TCP give the slow start threshold, *ssthresh*, at this cut?

- A. 50 segments, half the window at the loss, so *cwnd* goes on in congestion avoidance.
- B. 100 segments, the window at the loss, so slow start can bring *cwnd* back up to it.
- C. It keeps its value from the start, since only a timeout changes the threshold.
- D. 1 segment, since each loss event sets the threshold back to the initial window.

Correct answer: **A**

Why: At every loss event, three duplicate ACKs or a timeout, TCP sets *ssthresh* to half the window at the loss, here 50 segments. After three duplicate ACKs *cwnd* is halved to the same value, so *cwnd* sits at the threshold and grows by additive increase. A timeout resets *cwnd* to the initial window, while *ssthresh* still takes half the old window.

Q41. [MCQ]

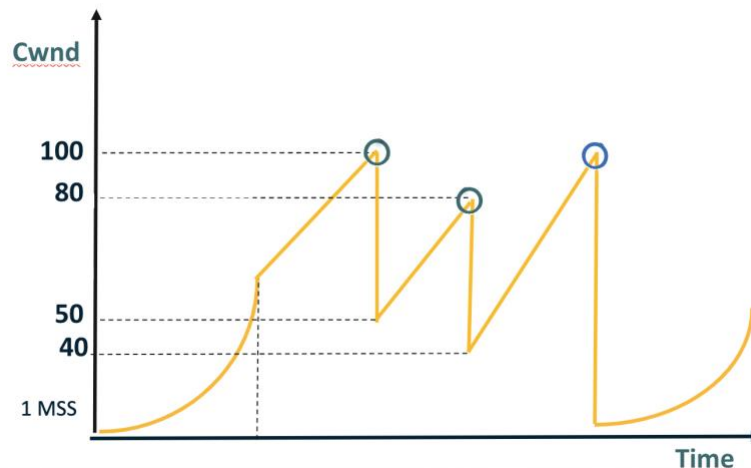


Figure: TCP *cwnd* over time (Module 2)

The figure shows the congestion window, *cwnd*, of a TCP sender over time. A timeout occurs near the right edge of the plot.

Which rule decides when TCP uses slow start and when it uses congestion avoidance?

- A. Slow start below the old peak of the window, and congestion avoidance above that peak.
- B. Slow start while *cwnd* is below *ssthresh*, and congestion avoidance once *cwnd* reaches it.
- C. Slow start only when the connection opens; after a timeout TCP uses congestion avoidance.
- D. Slow start right after each loss event, and congestion avoidance between loss events.

Correct answer: **B**

Why: Slow start doubles *cwnd* each round trip until it reaches the slow start threshold, and TCP then switches to additive increase. The threshold is half the window at the last loss. Slow start also runs after a timeout, which is the second exponential curve in the figure. After three duplicate ACKs TCP halves the window and continues in congestion avoidance.

Q42. [MCQ]

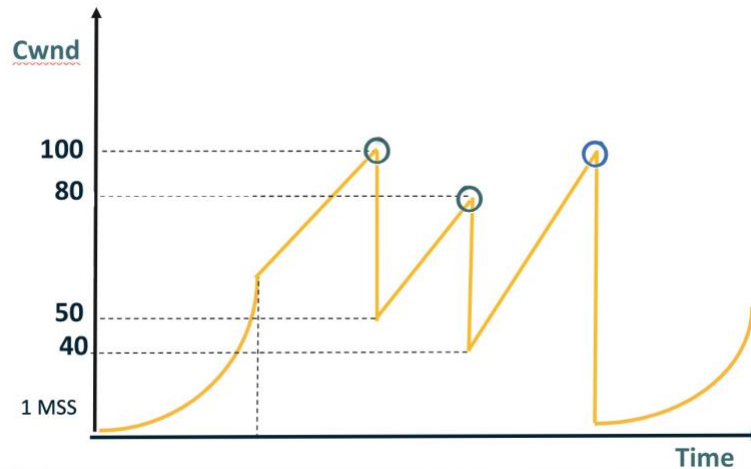


Figure: TCP cwnd over time (Module 2)

The figure shows the congestion window, cwnd, of a TCP sender over time. Again and again, cwnd climbs to a peak, such as 100, and is then cut back.

Why does cwnd not keep growing?

- **A.** Because cwnd reaches rwnd, and the full receive buffer then stops the sender until it drains.
- **B.** Because each time cwnd crosses ssthresh, TCP halves it and starts the climb over again.
- **C.** Because cwnd grows until a queue overflows and a packet is lost, and the loss cuts cwnd.
- **D.** Because routers send ICMP source quench when they fill, and TCP cuts cwnd on each one.

Correct answer: **C**

Why: TCP probes by raising its window until congestion begins. When a router's buffer fills, it drops a packet; the sender detects the loss, through three duplicate ACKs or a timeout, cuts cwnd, and starts the climb again. Reaching rwnd caps sending without cutting cwnd, and crossing ssthresh only changes the growth rule. ICMP source quench is no longer used.

Slow start in TCP

Q43. [TF]

A new TCP connection starts in slow start with a congestion window of cwnd = 1 segment. No loss occurs.

The congestion window grows by one segment per RTT, so the window reaches 4 segments after three RTTs.

- True
- False

Correct answer: **False**

Why: Slow start doubles cwnd every round trip, 1, 2, 4, 8, so after three round trips cwnd is 8 segments. Growth of one segment per round trip is additive increase, used only above the slow start threshold. The name refers to the small starting window.

Q44. [MCQ]

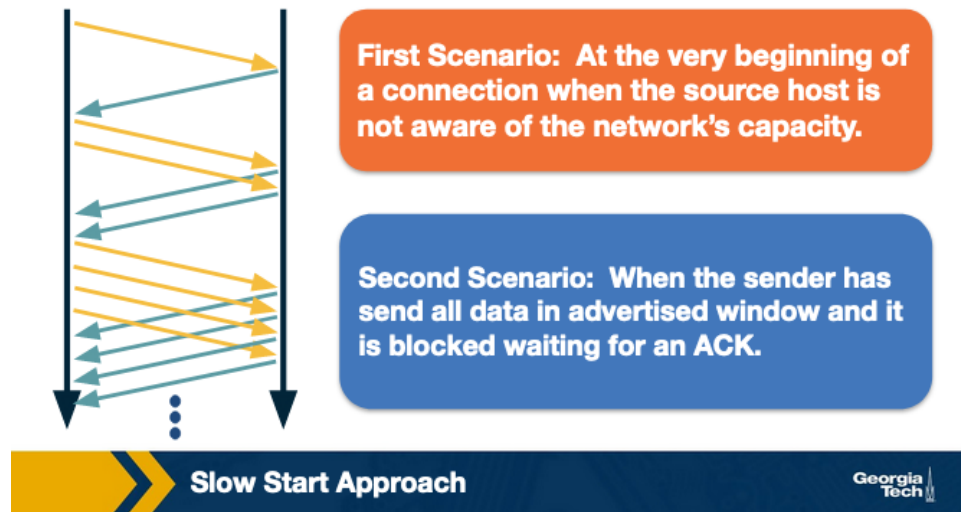


Figure: Slow start approach (Module 2)

The figure shows a TCP sender in slow start. The sender sends 1 packet, then 2, then 4, so its congestion window, cwnd, doubles each round trip.

What makes cwnd double each round trip?

- **A.** The sender doubles cwnd once per round trip on a timer, whatever number of ACKs arrive.
- **B.** The RTT halves each round as queues drain, so twice as many packets fit in a round trip.
- **C.** The receiver doubles its rwnd each round trip, and the sender sets cwnd equal to rwnd.
- **D.** Each ACK adds one packet to cwnd, and each packet sent in a round trip returns one ACK.

Correct answer: **D**

Why: In slow start, each acknowledgment adds one packet to cwnd. A round trip returns one ACK for each packet sent, so a window of 2 packets earns 2 ACKs and grows to 4. The growth comes from counting ACKs, with the RTT unchanged and no doubling timer. rwnd reports the free space in the receiver's buffer and leaves cwnd to the sender.

TCP Fairness

Q45. [TF]

Two TCP connections share a bottleneck link of capacity R . The two connections have the same RTT. Both connections adjust their windows with AIMD.

Over time, the throughput of each connection converges toward $R/2$.

- True
- False

Correct answer: **True**

Why: With the same RTT, both connections grow their windows at the same rate. Multiplicative decrease takes more from the larger one at each loss, so the gap closes over repeated cycles. Each settles near an equal share, $R/2$ of the link.

Q46. [MCQ]

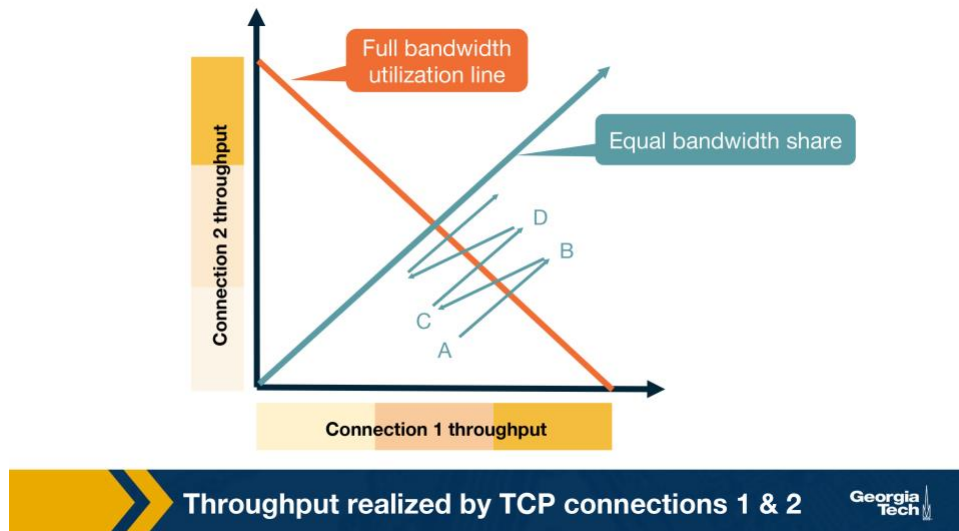


Figure: AIMD convergence to fairness; points A to D are successive operating points (Module 2)

Two TCP connections share a link of capacity R . The two connections have the same RTT. The figure plots the throughput of connection 2 against the throughput of connection 1. Under AIMD, the operating point of the two connections moves toward the intersection of the equal bandwidth share line and the full bandwidth utilization line.

Why does AIMD move the operating point toward this intersection?

- A. Additive increase keeps the gap between the two; multiplicative decrease shrinks it each time.
- B. Additive increase adds more to the smaller connection; multiplicative decrease keeps the gap.
- C. Routers tell each sender how far to slow down, and that feedback brings both to an equal share.
- D. Additive increase shrinks the gap; multiplicative decrease cuts both by the same amount.

Correct answer: **A**

Why: Additive increase moves both connections by the same absolute amount, so the gap between them stays the same. Multiplicative decrease cuts each by half, which takes more from the larger one, so every decrease shrinks the gap. Repeated cycles walk the pair toward the equal share line, with no signal from the routers.

Caution About Fairness

Q47. [MCQ]

Several standard TCP connections share one bottleneck link. Each connection belongs to a different application. The connections do not all get an equal share of the link.

Which connections take more than their fair share?

- **A.** Connections that started first, since a later connection halves its window on each loss.
- **B.** Connections with a short RTT, since their ACKs return sooner and their windows grow faster.
- **C.** Connections that see more losses, since they retransmit more of their data than the others.
- **D.** Connections with a long RTT, since each ACK covers more of the data they have in flight.

Correct answer: **B**

Why: Standard TCP grows its window on each arriving ACK, and ACKs return sooner on a short path. Short-RTT connections therefore take more than their share, and long-RTT connections get less. Each loss cuts a connection's window, so more losses mean less throughput. Every standard TCP connection halves its window on loss, whenever it started.

Q48. [TF]

A link carries several TCP flows and one UDP flow. The UDP flow sends at a fixed high rate. The link becomes congested.

The UDP flow slows down along with the TCP flows, so each flow keeps a similar share of the link.

- True
- False

Correct answer: **False**

Why: UDP has no congestion control, so the UDP sender keeps its rate when the link congests. The TCP flows slow down, and the UDP traffic takes the capacity they give up. TCP's fairness holds only among senders that cooperate by slowing down.

Congestion Control in Modern Network Environments: TCP CUBIC

Q49. [MCQ]

TCP CUBIC sets its congestion window with the growth function $W(t) = C(t - K)^3 + W_{\max}$.

What does the variable t represent in this function?

- **A.** The number of ACKs received since the last loss, as in standard TCP's window growth.
- **B.** The current RTT estimate, which TCP updates from the samples it measures.
- **C.** The time elapsed since the last loss event, as measured by the sender's clock.
- **D.** The time since the connection opened, so $W(t)$ grows the longer a flow lasts.

Correct answer: **C**

Why: t is the time elapsed since the last loss event, measured by the sender's clock. The clock restarts at each loss, so the curve starts again from the reduced window after every loss, whatever the connection's age. Standard TCP grows per ACK; CUBIC's use of time is what lets flows with different RTTs grow alike.

Q50. [MCQ]

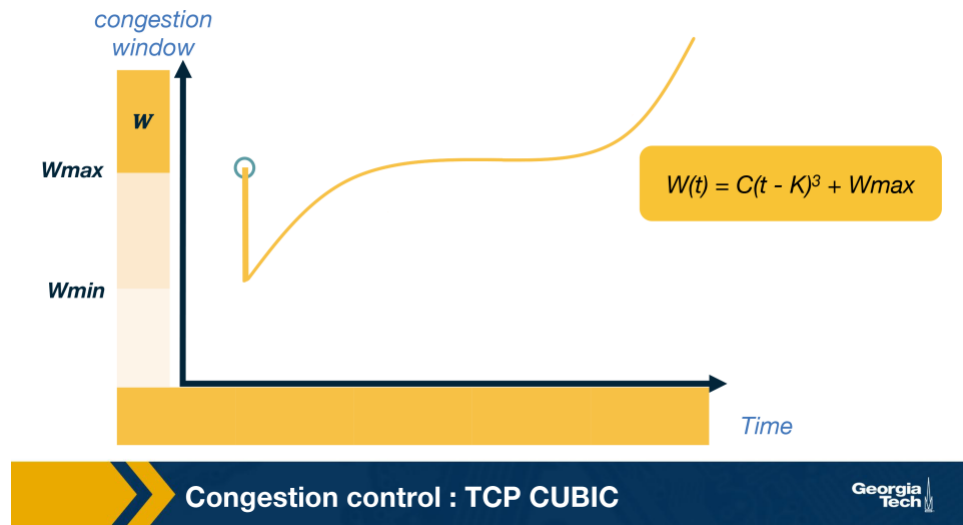


Figure: TCP CUBIC window growth function (Module 2)

The figure shows how TCP CUBIC grows its congestion window over time. Far below W_{max} , the window grows quickly. Near W_{max} , the window grows slowly.

Why does CUBIC grow its window quickly far below W_{max} but slowly near W_{max} ?

- **A.** Because the receiver advertises a smaller $rwnd$ as $cwnd$ nears W_{max} , which slows the sender.
- **B.** Because CUBIC grows once per ACK, and fewer ACKs come back as queues build near W_{max} .
- **C.** Because RTT samples become noisy near W_{max} , so CUBIC pauses until its estimate settles.
- **D.** Because the last loss occurred at W_{max} , which makes CUBIC cautious as it nears W_{max} .

Correct answer: **D**

Why: W_{max} is the window at which loss was detected. Far below it the connection ran before without trouble, so CUBIC grows quickly; near it, CUBIC grows cautiously. The curve is a function of the time since the last loss alone, so ACK arrivals, RTT samples and $rwnd$ leave its shape unchanged.

Q51. [MCQ]

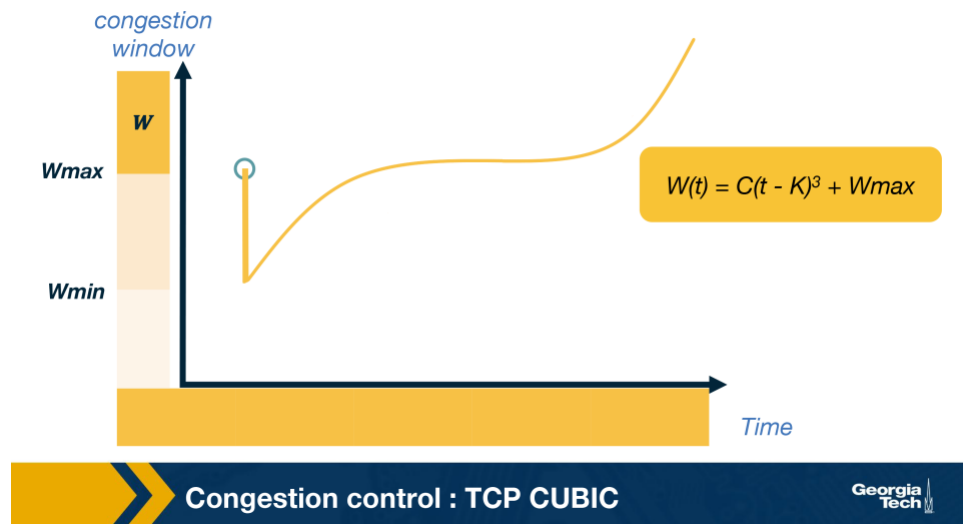


Figure: TCP CUBIC window growth function (Module 2)

The figure shows how TCP CUBIC grows its congestion window over time. W_{max} is the window at which the last loss occurred. The window grows back to W_{max} . No new loss occurs. The growth then speeds up again above W_{max} .

Why does CUBIC speed up its growth above W_{max} ?

- **A.** Above W_{max} , CUBIC returns to slow start, since the window has passed its threshold.
- **B.** The old loss may have been transient, or a flow that left may have freed capacity.
- **C.** Queues drain above W_{max} , so ACKs return sooner and each ACK adds more to the window.
- **D.** The receiver may raise $rwnd$ above W_{max} , and CUBIC grows to fill the larger receive window.

Correct answer: **B**

Why: W_{max} is the window at which the last loss occurred. If the window passes it with no new loss, the earlier loss may have been transient, or a departing flow may have freed capacity, so CUBIC probes for more. The speed-up comes from the cubic growth function itself, which depends only on the time since the last loss; slow start, faster ACKs and a larger $rwnd$ do not enter it.

Q52. [MCQ]

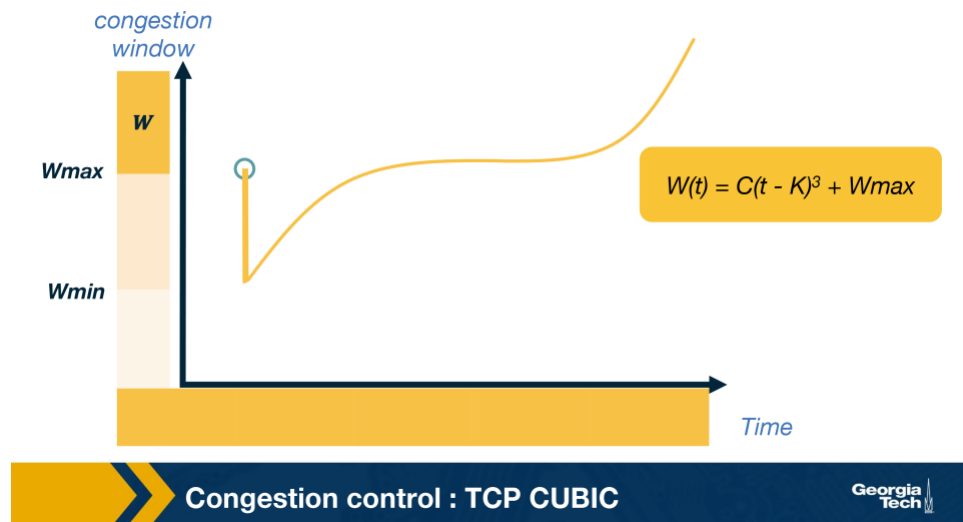


Figure: TCP CUBIC window growth function (Module 2)

TCP CUBIC sets its congestion window with the growth function $W(t) = C(t - K)^3 + W_{max}$. CUBIC is RTT-fair, and standard TCP is not.

Which property of the growth function makes CUBIC RTT-fair?

- A. $W(t)$ grows once per ACK, and the flows' ACKs share the bottleneck's return path equally.
- B. CUBIC keeps 80 percent of its window on a loss, so no flow falls far behind the others.
- C. $W(t)$ depends on clock time since the last loss, so flows grow alike whatever their RTTs.
- D. $W(t)$ counts round trips since the last loss, so each flow climbs the same curve per RTT.

Correct answer: C

Why: t is clock time since the last loss, so two CUBIC flows compute their windows from the same clock and grow at the same rate regardless of RTT. Growth per ACK or per round trip would favor the flow whose ACKs return sooner, which is the unfairness of standard TCP. CUBIC's gentler 0.2 reduction serves a different purpose: it keeps utilization high.

Q53. [MCQ]

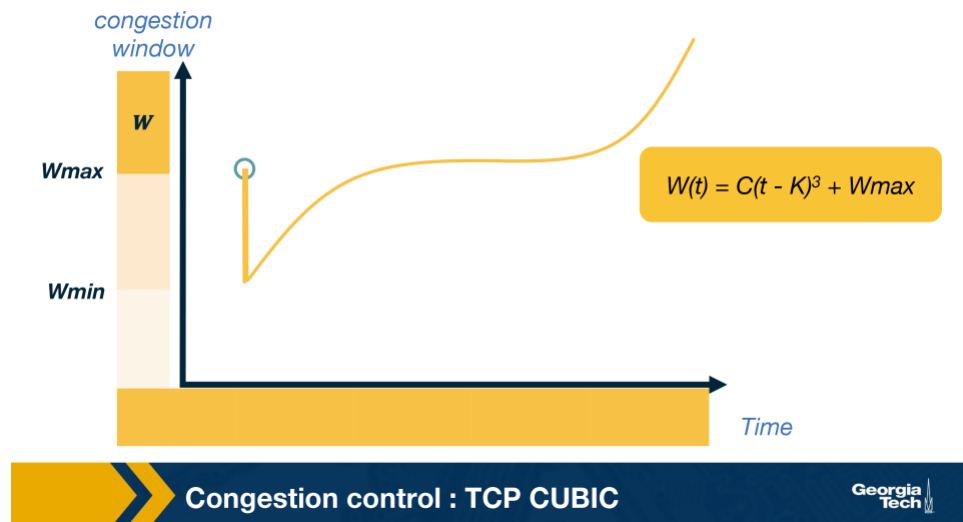


Figure: TCP CUBIC window growth function (Module 2)

TCP CUBIC grows its congestion window as a function of the time since the last window reduction, not of the number of RTTs. On a path with a short RTT, standard TCP would sometimes grow its window faster than CUBIC does. In that case, CUBIC switches to its TCP-friendly mode.

How does CUBIC operate in its TCP-friendly mode?

- A. It switches to standard TCP's halving on loss, but keeps CUBIC's curve between losses.
- B. It pauses cwnd growth for one RTT so the standard TCP flows on the link can catch up.
- C. It caps CUBIC's window at standard TCP's, so CUBIC takes no more than standard TCP would.
- D. It tracks the window standard TCP would have, and uses that window when it is larger.

Correct answer: **D**

Why: On short-RTT paths, a window grown on a fixed clock could be slower than standard TCP. CUBIC therefore tracks the window standard TCP would have and uses it whenever it is larger; this is its TCP-friendly mode. The tracked window acts as a floor under CUBIC's window, so CUBIC keeps growing at least as fast as standard TCP, and its reduction on loss stays the same.

The TCP Protocol: TCP Throughput

Q54. [TF]

A simple model of TCP throughput is derived from the sawtooth pattern of the congestion window. The model rests on a short list of assumptions.

One of the assumptions of the model is that losses are detected by retransmission timeouts.

- True
- False

Correct answer: **False**

Why: The model assumes losses are detected by triple duplicate ACKs, with no timeouts. A timeout would reset the window to its initial size. That would break the regular sawtooth the derivation measures, in which the window is halved at W and climbs back by one packet per RTT. The other assumptions are a single long-lived connection in steady state with no slow start, a constant RTT, a constant maximum window W , and exactly one loss per cycle, evenly spaced. In practice, timeouts are one of the factors that push the constant C below 1.

Q55. [TF]

Two long-lived TCP connections have the same RTT and the same MSS. The loss probability on the path of the first connection is 100 times the loss probability on the path of the second connection.

Under the TCP throughput model, the throughput of the first connection is one tenth of the throughput of the second connection.

- True
- False

Correct answer: **True**

Why: In the model, throughput is proportional to MSS divided by RTT, times one over the square root of the loss probability. With MSS and RTT equal, a loss rate 100 times higher divides the throughput by the square root of 100, which is 10.